Bilkent University

Department of Electrical and Electronics Engineering

Term Project Report: Smart Posture Monitor and AFK Alert System

Video Link: https://youtu.be/zX3cPm8u5OE

Purpose

The main objective of this final term project is to create a system that helps users to monitor their posture by using a Basys3 FPGA board, an HC-SR04 ultrasonic sensor, a photoresistor (LDR), and a breadboard with several jumpers to establish the structure of the system. Sometimes, users do not realize their bad postures while working in front of the screen, so this system aims to prevent users from their bent postures by measuring the distance between the user and the screen continuously and giving them warning feedback visually initially. If they somehow do not realize the warning and still fail to correct their postures within this given time, the system uses a buzzer to give an auditory warning. Moreover, this system decides this timer by first looking at the LDR to avoid eye strain. Consequently, the project demonstrates to us a great application of finite state machines (FSM) and sequence detector logic, and while designing the project, the system uses VHDL programming on an FPGA without relying on unnecessary components; nevertheless, the structure of the system contains some external components to create a final professional product.

Design Specifications

The general structure of the system is built with hardware constraints and hardware logic on them. The heart of this system is our Basys3 board since we use other components to activate and decide how they behave through this advanced development board. It is also connected to the other components such as the HC-SR04 ultrasonic sensor and a 5 mm LDR through its PMOD pins. A voltage divider is used in the circuit for the LDR to provide logic levels while giving the system information on whether the environment is dark or bright. The circuit uses a 10K Ohm resistor in the process. Additionally, a 5V active buzzer is integrated into the breadboard for auditory feedback in case of any violation of the threshold and visual warning.

The system operates above the predetermined 40 cm safe distance threshold. When the distance is measured below this intended threshold, the FSM triggers a countdown timer. The duration of this timer is adjusted based on the brightness of the environment. This duration is set 10 seconds for bright environments and 5 seconds for dark environments, considering the increased eye strain in those areas.

To briefly explain how buzzer gives auditory feedback, if the user fails to correct the posture in this given time, which is adjusted depending on the environment, the system enters a

“LOCKED State”. In this state, the external buzzer and the built-in LED on the Basys3 FPGA board are activated, but the external LED is turned off because the “COUNTDOWN State” is no longer active to warn the user visually. To silence the alarm and get out of this state to be in the “SAFE State”, a specific password (“Up -> Down -> Left”), which is created by the developer, must be entered correctly using the directional buttons on the board; otherwise, the system will get stuck in the same state and will maintain its auditory feedback until the password is given. This is basically like a hardware sequence detector.

For visual feedback, the system uses the 4-digit 7-segment display, which is coded by a multiplexer circuit, to illustrate the current distance to the user during the “SAFE State” to help them adjust the intended threshold before spending time on the screen. However, if the measured distance falls below this pre-determined distance, which is 40 cm, the new state will be the “COUNTDOWN State”, and the 4-digit 7-segment display is going to show the timer decided by the environmental light. If the user does not correct the posture, the 4-digit 7-segment display will just show “0” in the “LOCKED State” unless the password is entered.

To make the system clean and organized for the user, I decided to build a custom box myself. Basically, this box is like a stand for the small components of the system and makes the product much cleaner and organized. It prevents a cable mess because there is a vast amount of jumper cables which do not reflect a clean image. Furthermore, the HC-SR04 sensor, the LDR, and the external LED are connected to the breadboard, but I lengthened them with wires and positioned them outside of the box for a properly working design. This external LED gives a direct visual warning when the countdown timer starts, and the user is in the danger zone for eye strain. The overall structure of the system would be much more user-friendly to catch the person’s attention instantly.

Methodology

The system’s overall design is strictly modular. I wrote this code using VHDL on the Vivado software. Instead of writing huge and complicated code, which is impossible to debug, I divided this project into some sub-modules to make a clean hierarchy for my convenience. While dividing the code, I used four fundamental sub-modules, and I compiled all the sub-modules together inside a *top_module*. This *top_module* works as a roof of the system that keeps the main structure.

1. **Clock Divider:** Normally, the Basys3 board has an internal *system_clock* which runs at 100 MHz. However, the different components of the project need different clock speeds to work properly. Therefore, I wrote this module to slow down the main clock because this original *system_clock* of the Basys3 ends before the system starts. It uses counters to generate a 1-microsecond pulse (*tick_1us*) for the ultrasonic sensor and a 1-second pulse (*tick_1sec*) for the countdown timer. *(AI Usage Note: I used Google Gemini AI to double-check my mathematical calculations for finding the correct counter limits to divide the 100 MHz system clock.)*

2. **Ultrasonic Sensor Controller:** This module controls the HC-SR04 sensor using a small state machine. At first, it sends a 10-microsecond high pulse to the *sensor_trigger* pin. Then, it waits for the *sensor_echo* signal and counts how many microseconds it stays high. Because the speed of sound in the air is constant, the module divides this total time by 58 to accurately calculate the exact distance in centimeters. It frequently sends this calculated distance to the main FSM of the system. *(AI Usage Note: I watched many tutorials to understand how the HC-SR04 hardware works, but I got coding support from Gemini AI to correctly build the 10-microsecond trigger and echo measurement states in VHDL.)*
3. **Master Finite State Machine (FSM):** This module is the main brain of the project. It evaluates the distance, the ambient light (*ldr_input*), and the button presses to control the system. The FSM operates in three main states:
    - **SAFE State:** This is the default state of the system. If the measured distance is 40 cm or greater, the system stays here. The 7-segment display shows the real-time distance. All alarms and the external LED are turned off in this state.
    - **COUNTDOWN State:** If the user gets closer than 40 cm, the FSM changes to this state. The external LED becomes on immediately. The system first checks the LDR input initially and starts a countdown timer. If the user moves back to a safe distance (>= 40 cm) before the timer reaches 0, the system forgives the user and returns to the SAFE state back.
    - **LOCKED State:** If the countdown timer reaches to 0, the system locks itself. The display shows “0”, the built-in warning LED turns on, and the buzzer rings continuously unless the password is entered correctly by the user.

    To represent the Master FSM in a formal digital logic format, the system states and external inputs are assigned to some specific binary values.

    - **State Variables (S_1S_2):** SAFE = "00", COUNTDOWN = "01", LOCKED = "10"
    - **Input D (Distance):** '1' if distance < 40 cm (Violation), '0' if distance ≥ 40 cm (Safe).
    - **Input T (Timer):** '1' if the countdown timer reaches 0, '0' if timer > 0.
    - **Input P (Password):** '1' if the correct sequence is entered, '0' otherwise.
    - **Outputs (A, E):** Alarm/Buzzer = A, External Red LED = E.

Present State (S_1S_2)	**D**	**T**	**P**	**Next State ($S_1 + S_2 +$)**	**Alarm Output (A)**	**Ext. LED Output (E)**
00	0	X	X	00	0	0
00	1	X	X	01	0	0
01	0	X	X	00	0	1
01	1	0	X	01	0	1
01	1	1	X	10	0	1
10	X	X	0	10	1	0
10	X	X	1	00	1	0

Table 1: Master FSM State Transition and Output Table



Figure 1: State Transition Diagram for The Master FSM

The system adjusts the timer depends on the room light to prevent from eye strain. The brief logic is illustrated in the truth table below:

Distance	ldr_input (Light)	FSM State Transition	Timer Duration
>= 40 cm	Don't Care	Remains in SAFE	None (Shows Distance)
< 40 cm	1 (Bright)	Moves to COUNTDOWN	10 Seconds
< 40 cm	0 (Dark)	Moves to COUNTDOWN	5 Seconds

Table 2: Dynamic Timer Truth Table

4. **Sequence Detector (Password Logic):** To get out of LOCKED state, I implemented a sequence detector logic. I didn't want the user to just mash buttons randomly to stop the alarm. So, I implemented a sequence detector logic. The user must press the directional buttons in a specific order: *button_up*, then *button_down*, then *button_left*. The code uses edge detection, so the user cannot just hold the buttons down.
    - **Step 0:** The system waits. If Up pressed, it goes to Step 1.
    - **Step 1**: If Down is pressed, it goes to Step 2. If any other button is pressed, it resets to step 0.
    - **Step 2:** If Left is pressed, the password is correct. The system unlocks and returns to the SAFE state. Any wrong input resets the sequence.

    It basically works as a counter to hold the memory of the password.

5. **Seven-Segment Display Driver:** The 4-digit display on the Basys3 board shares the same cathode pins. Because the 4-digit display on the Basys3 board shares the exact same cathode pins, we cannot just send different numbers to all four digits normally. This means we cannot send different numbers to all four digits at the exact same time. To solve this, this module uses a multiplexer circuit. It uses a fast refresh counter to turn the digits on and off one by one very quickly. Because of the persistence of vision, the human eye sees all digits as turned on at the same time. The module also uses modulo (*mod 10*) and division operations to separate the digits of the distance or the timer to show them on the screen.
6. **Boolean Logic and Edge Detection Equations:** To ensure the hardware works stable, specific Boolean logic equations are used in the design. For example, to prevent the user from holding the password buttons down and breaking the sequnece, an “Edge Detecion” logic is implemented. *(AI Usage Note: I asked Gemini AI how to stop the system from accepting long button holds. The AI suggested using this edge-detection logic with previous button states.)* This logic uses logical AND (·) and NOT ($\bar{x}$) operators to detect only the exact moment the button is pressed. If B is the current button state and B_{prev} is the previous state, the rising edge (E) equation is:

$$E = B \cdot \overline{B_{prev}}$$

This Boolean logic is applied to all three password buttons in the VHDL code:

$$UP_{edge} = UP \cdot \overline{UP_{prev}}$$

$$DOWN_{edge} = DOWN \cdot \overline{DOWN_{prev}}$$

$$LEFT_{edge} = LEFT \cdot \overline{LEFT_{prev}}$$

Also, the external red LED is controlled by a simple Boolean condition based on the Master FSM states. If we must represent the SAFE state as "00", COUNTDOWN as "01", and LOCKED as "10" using two state variables (S_1 and S_0), the external LED only turns on during the COUNTDOWN state. The logic equation for the external LED is:

$$LED_{ext} = \overline{S_1} \cdot S_0$$

These mathematical Boolean equations can be translated directly into VHDL logic to build the combinational parts of the FSM.

Results

The overall performance of the system is verified in two main stages. One of them is software simulation and analysis in Vivado. The other one is physical hardware testing on the Basys3 board.

1. Software Simulation and Vivado Analysis

First, I analyzed the design using Vivado's built-in tools to ensure the VHDL logic and connections were correct before flashing the code to the FPGA.

RTL, Synthesized, Implemented and Elaborated Designs

I used RTL analysis to visualize the structure of my project. Figure 2 shows how the main sub-modules are connected to each other inside a top module. After checking the RTL schematics, I ran the synthesis and opened the synthesized design as illustrated in Figure 4. This schematic basically shows us how my VHDL code is transformed into actual logic gates. Finally, I opened the implemented design (Figure 6). This figure clearly demonstrates how the logic gates and VHDL code are integrated into the Basys3 FPGA chip physically. I also put all of these three elaborated versions in the figures. These three visuals prove that the structure between the sensor, the FSM, and the display driver is established successfully on the hardware aspect, as understood from the figures.

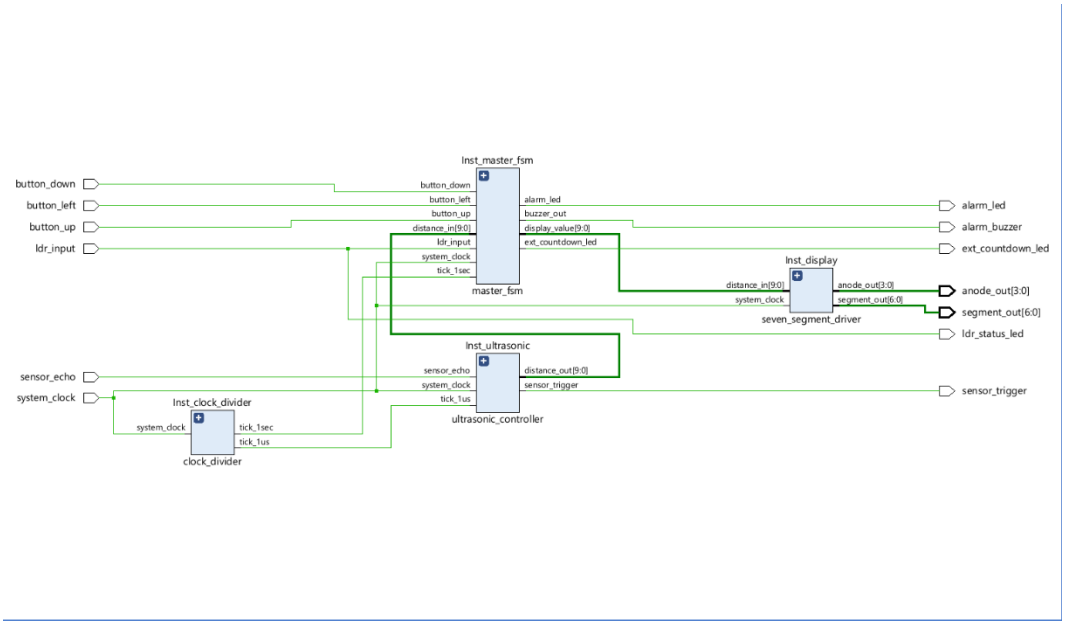


Figure 2: RTL Schematic of the Top Module

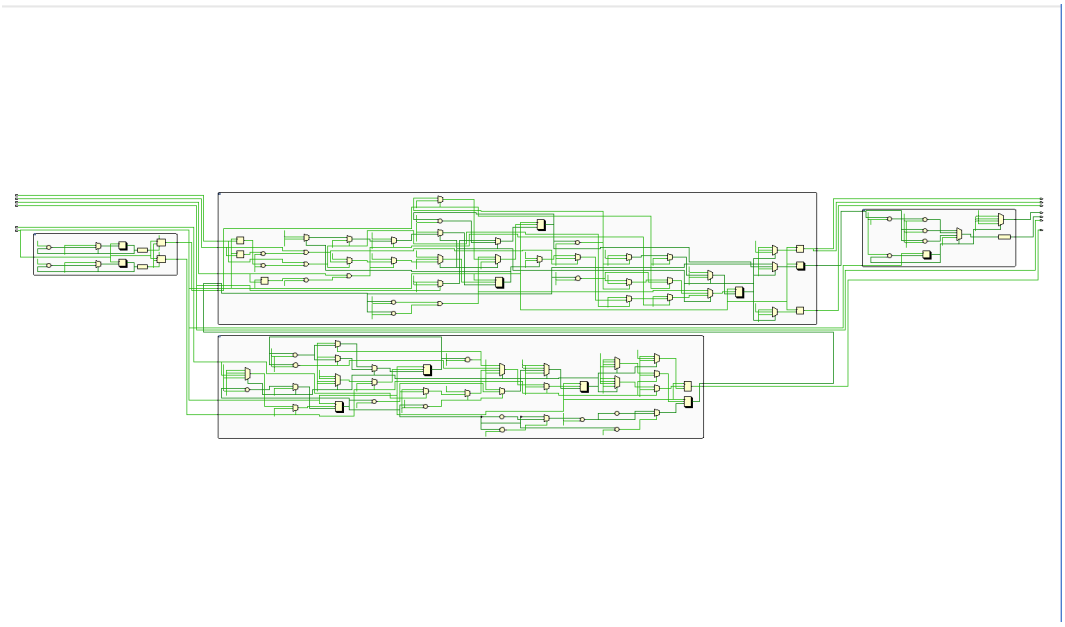


Figure 3: Elaborated RTL Schematic of the Top Module

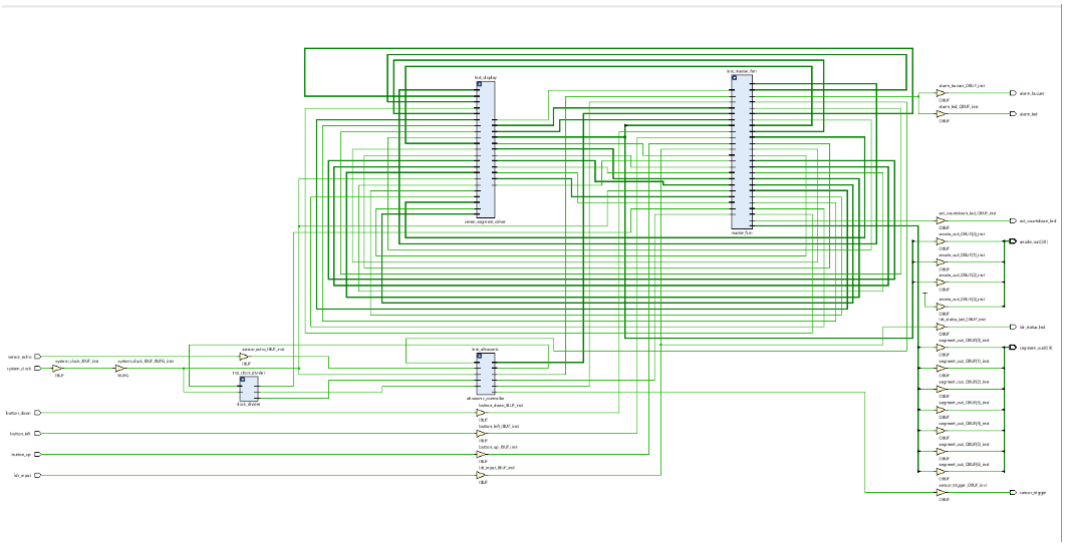


Figure 4: Synthesized Design Schematic

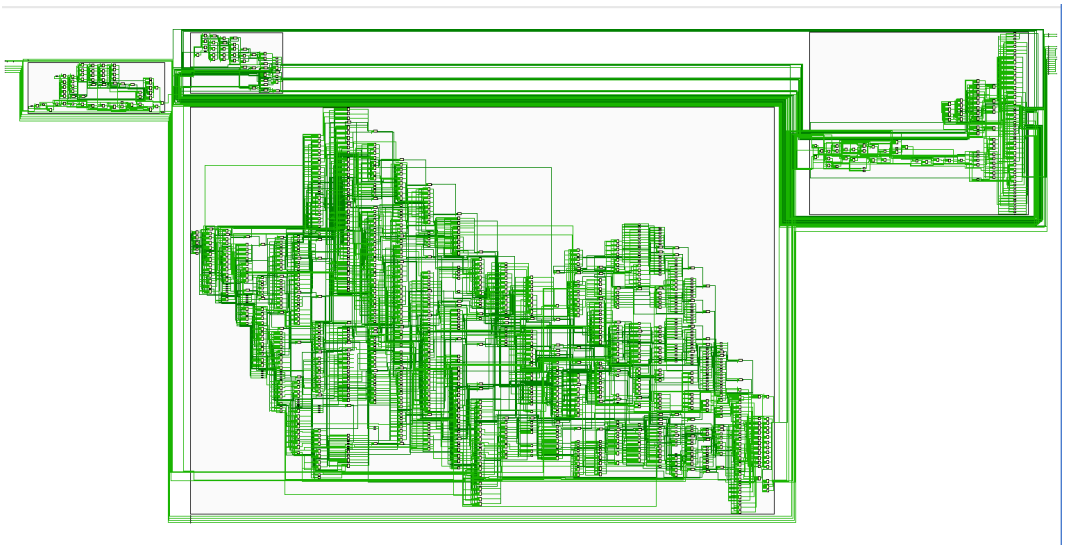


Figure 5: Elaborated Synthesized Design Schematic

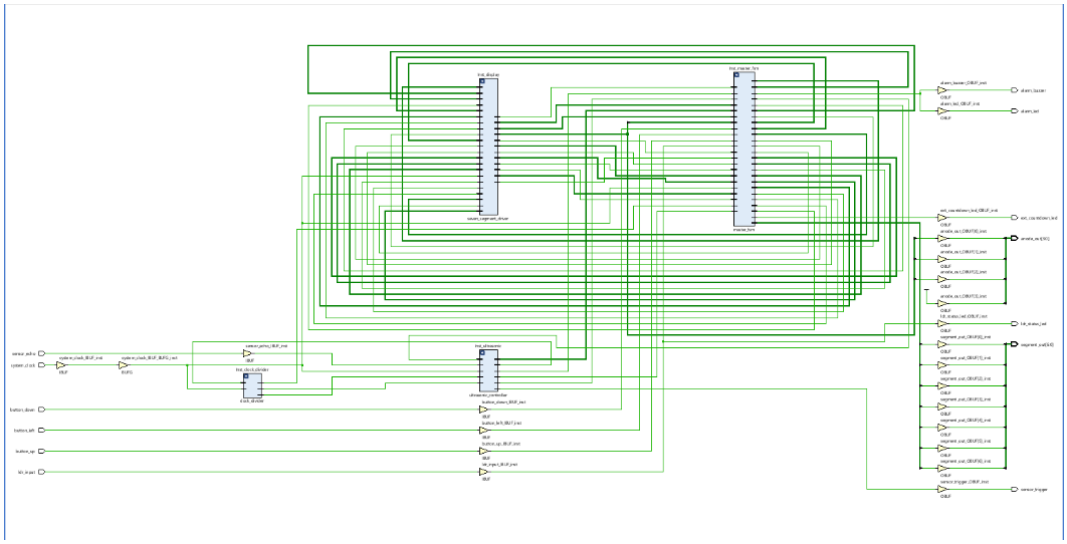


Figure 6: Implemented Design (Device Routing)

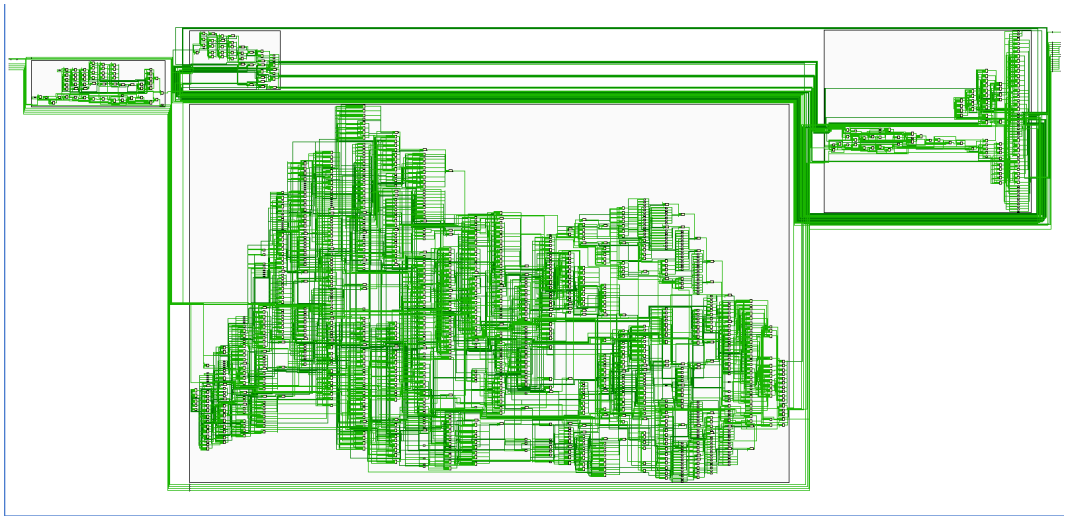


Figure 7: Elaborated Implemented Design

Simulation Waveforms

To verify the FSM Transitions, I performed a behavioral simulation. As seen in **Figure 8**, when the *distance_in* signal drops below 40, the *current_state* immediately changes from SAFE to COUNTDOWN. The simulation also proves to us that the *tick_1sec* pulse correctly decrements the timer. *(AI Usage Note: Simulating a physical sensor's delay in Vivado is very complex. Therefore, I used Gemini AI to help me write the testbench code that generates a fake echo signal for the simulation.)*

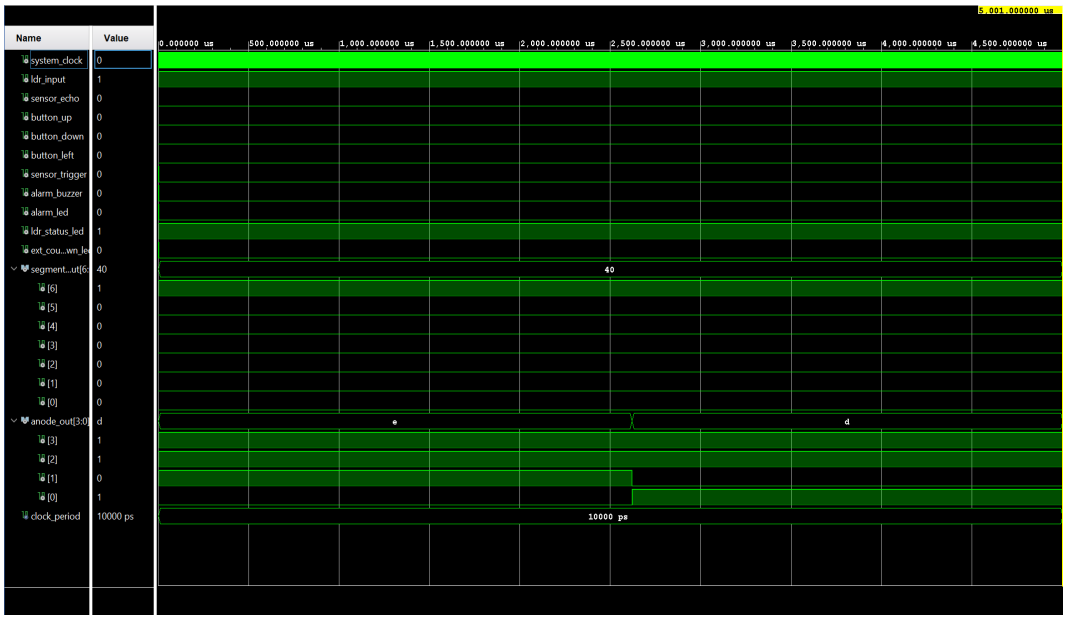


Figure 8: Simulation Waveforms Showing State Transitions

Resource Utilization and Timing

I checked the Utilization Report to see how much of the Basys3 resources were used. The project uses a very small percentage of the Look-Up Tables (*LUTs*) and Flip-Flops (*FFs*), proving the design is efficient.

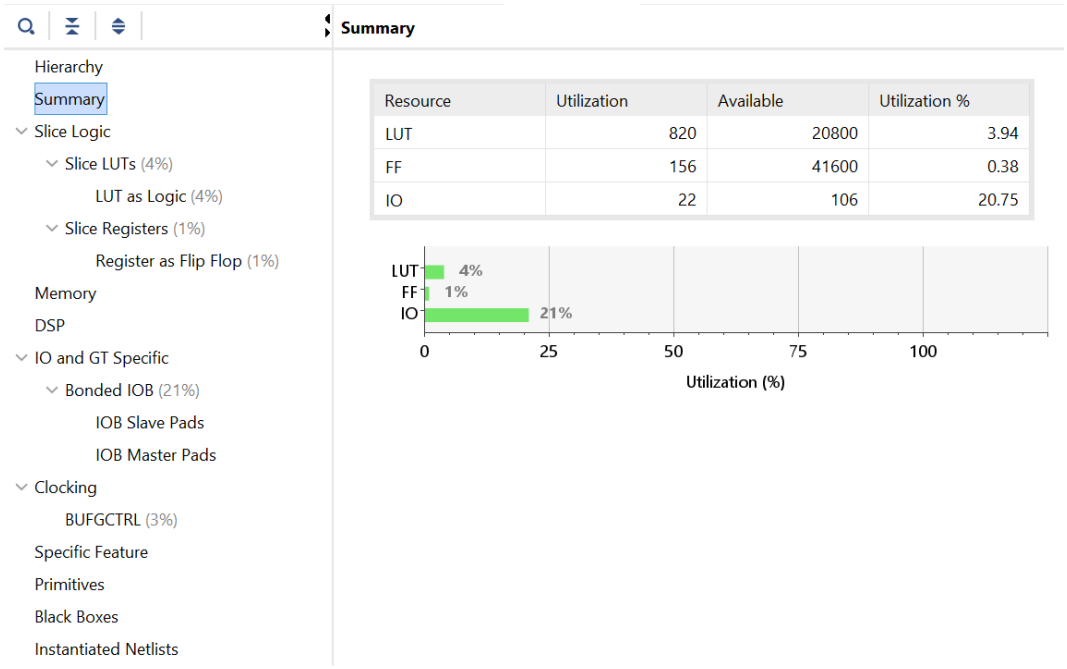


Figure 9: Resource Utilization Summary

Regarding timing, I observed a "Timing Violation" warning in the Timing Summary report. This is because I used division *(/ 58*) and modulo (*mod 10*) operators, which create long combinational paths for the FPGA. However, since the system operates at low human-interaction speeds, this did not affect the hardware performance.

2. Hardware Realization and Physical Testing

After verifying the software results, I tested the system on the Basys3 board inside my custom-built enclosure to observe and test the project physically.

Overall System Setup

Figure 10 shows the final product of the system. The Basys3 and the Breadboard are hidden inside the box to prevent cable mess which creates a clear workplace. The ultrasonic sensor and LDR are positioned to face the user with the external LEDs.

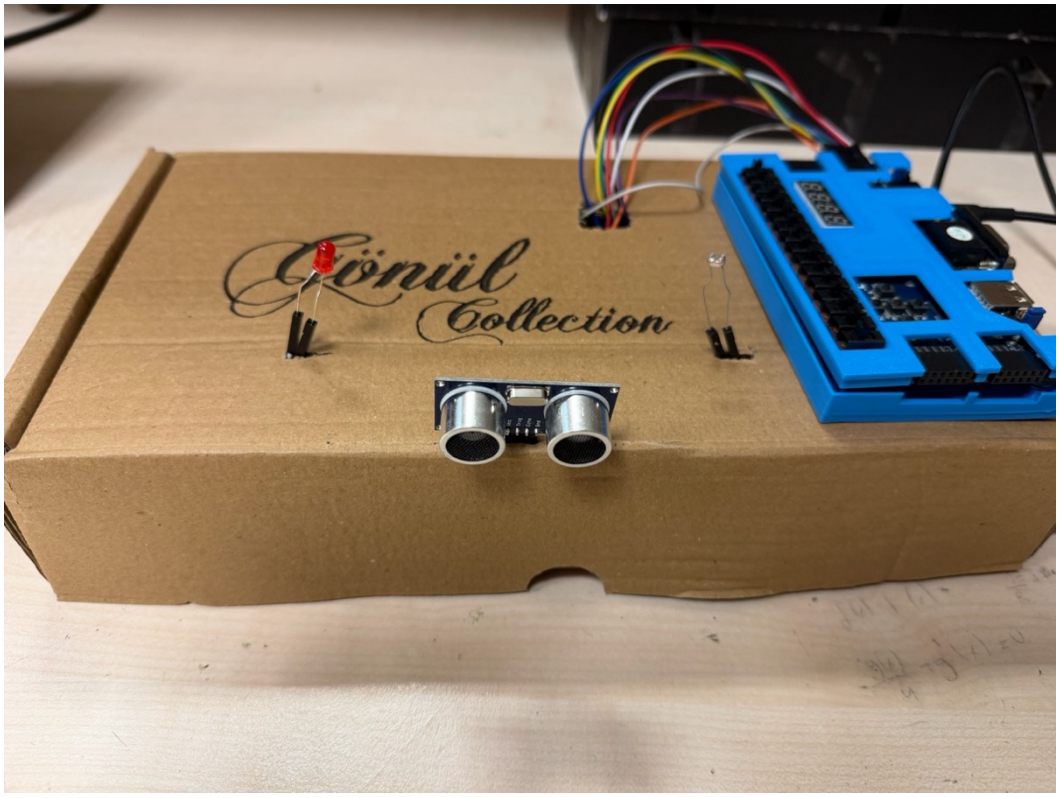


Figure 10: Final Product of the System

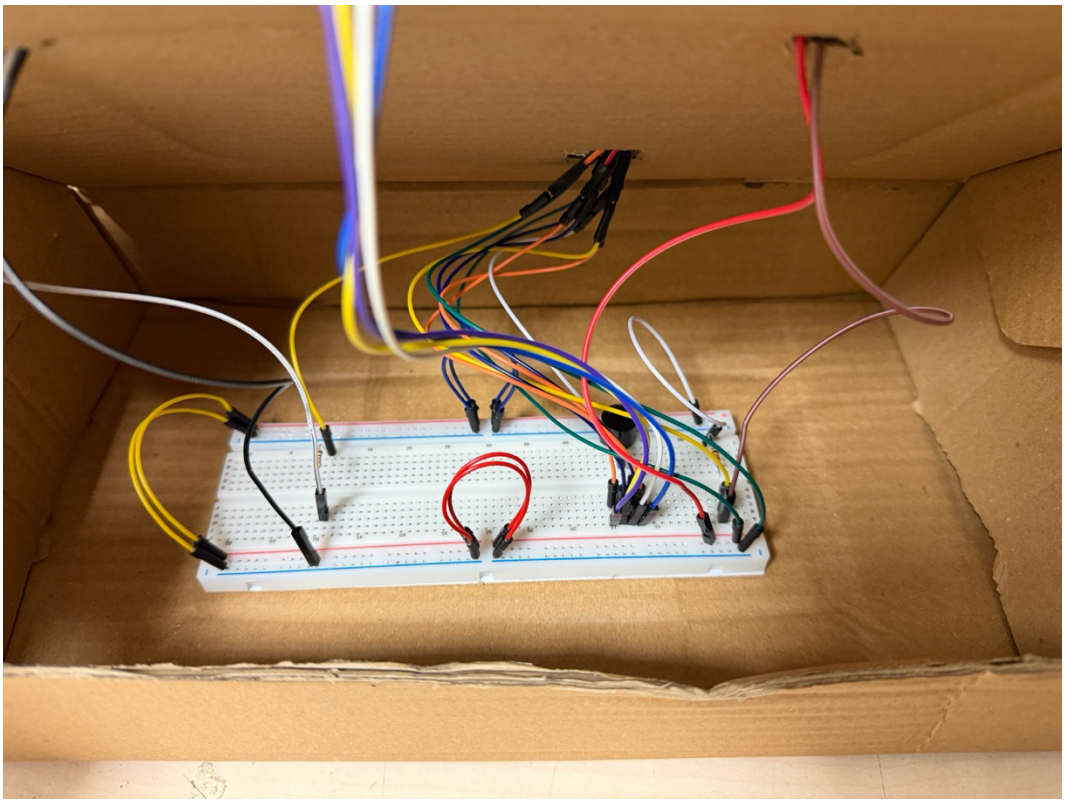


Figure 11: Inside of the Custom-Built Enclosure

Testing the SAFE State

During the SAFE state, the display shows the distance. In **Figure 12**, the user is at a safe distance (55 cm), and all alarms are off.



Figure 12: System is in SAFE State showing real-time distance

Testing the COUNTDOWN State and LDR Logic,

When the user moves closer than 40 cm, the system enters the COUNTDOWN state. The external red LED on top of the sensor turns on.

- In Figure 13, the room is bright, and the system starts a 10-second timer.
- In Figure 14, I covered the LDR to simulate a dark room, and the system dynamically started a 5-second timer.

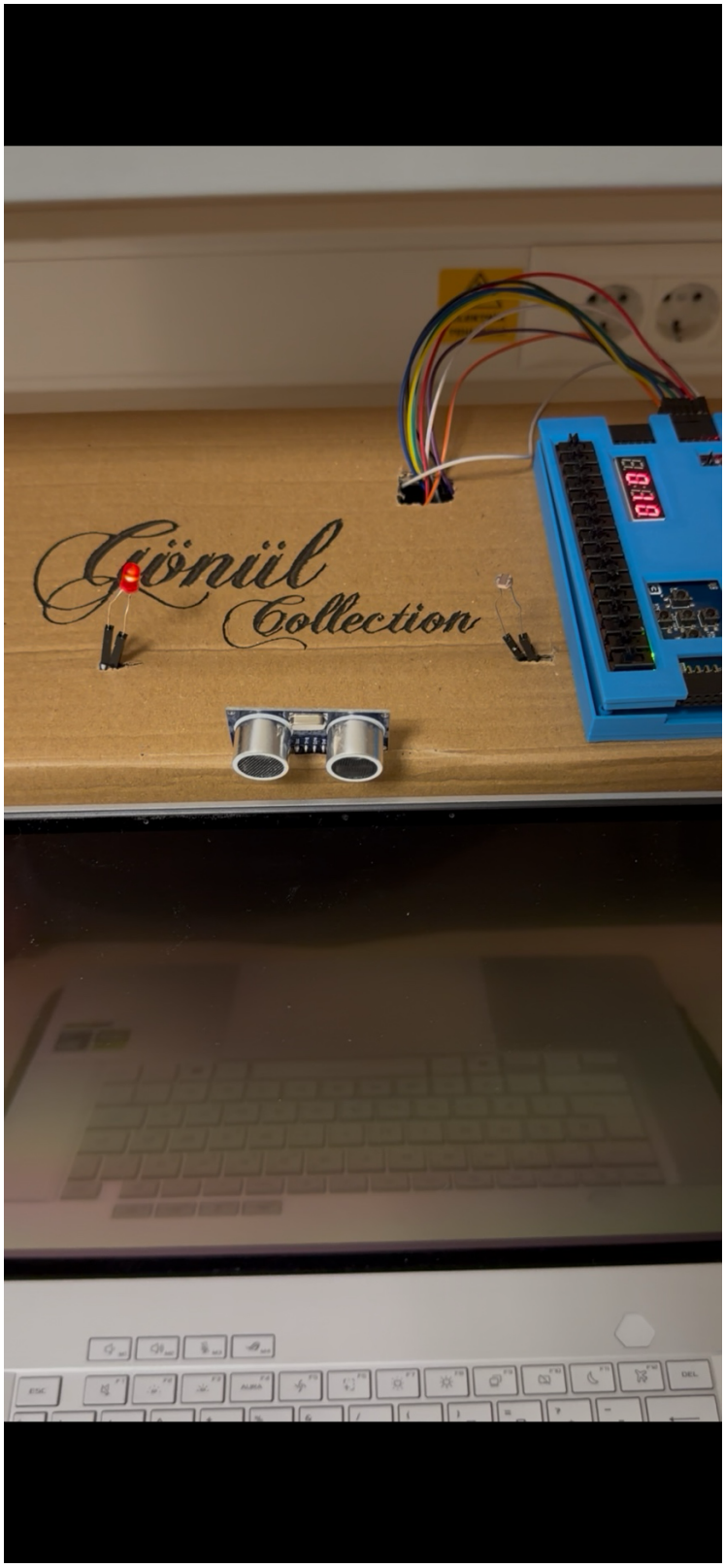


Figure 13: Countdown Starting from 10s in a Bright Environment



Figure 14: Dynamic 5s Countdown in a Dark Environment

Testing the LOCKED State and Password

If the user ignores the warning, the timer reaches 0, and the system enters the LOCKED state (**Figure E**). The internal LD0 LED and the buzzer are activated. The only way to reset the system is the Up -> Down -> Left sequence.

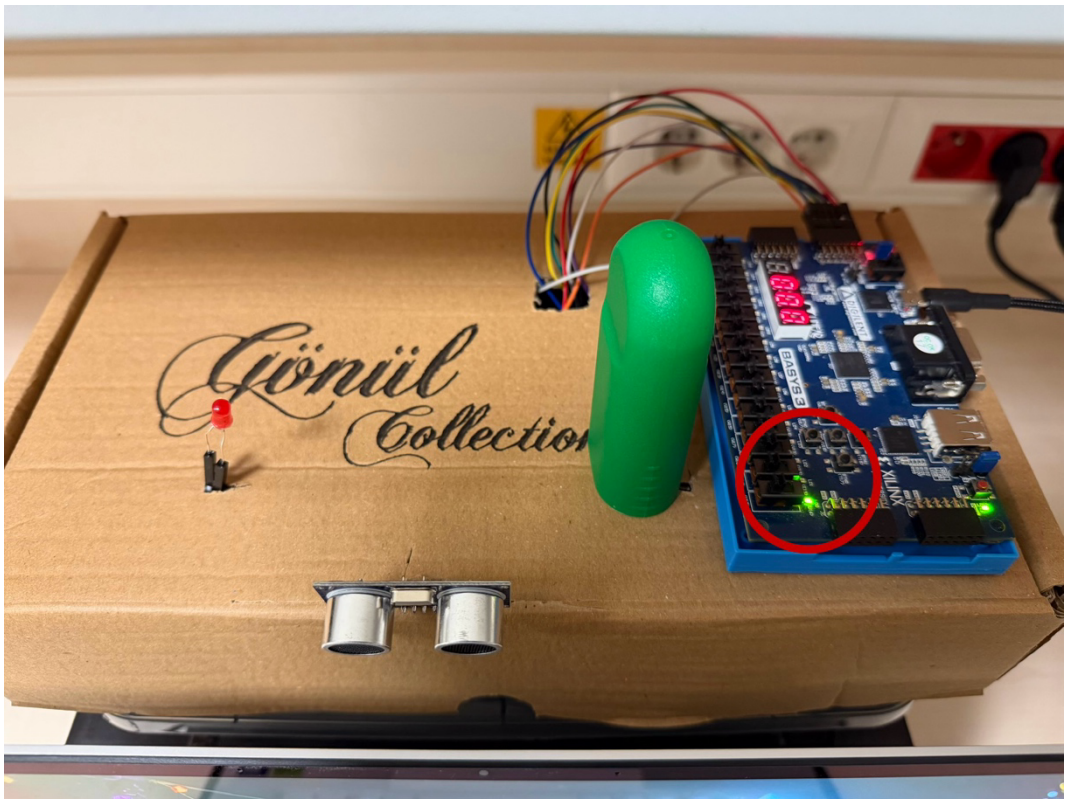


Figure 15: LOCKED State with active alarm and LD0 LED

Conclusion

In this final term project, I managed to create a Smart Posture Monitoring and AFK alert system successfully by using the Basys3 board and other components implemented on a breadboard. The main goal of the system was to design and implement a system that helps users correct their posture by measuring the exact distance between the user and the screen frequently. I achieved this goal by integrating an ultrasonic sensor, a photoresistor (LDR), and a 5V active buzzer on a custom box, and I used VHDL to establish the software of the system. This project was a great application of finite state machines (FSM). The system had flawless and stable transitions between these states. SAFE, COUNTDOWN, and LOCKED states worked properly as intended. One of the crucial things about the project was checking the environmental light input by using an LDR. The system automatically adjusted the timer when the room was dark to prevent the user from eye strain, which made this system especially versatile and user-friendly. Furthermore, the design successfully included a sequence detector for the unlocking mechanism for the

states. The user had to enter the “Up -> Down -> Left” sequence using the directional buttons on the Basys3 board. Also, adding edge cases to the code increased the security and reliability, such as what if the user pressed the buttons for too long. In terms of other hardware systems, using the seven-segment multiplexer gave clear feedback about the real-time distance and the countdown depending on the states. During the implementation phase, I encountered some challenging and frustrating challenges, especially some timing violations and syntax errors that Vivado gave me for no reason. The use of division and long lines of calculations, especially while triggering the ultrasonic sensor, gave warnings. I got a “Timing 38-282” warning in Vivado. When I searched it to learn why it happened, I learned that some modulo operations bugged the system. These operators like division were slow compared to the FPGA and caused setup time delays. However, there was the fact that my project interacted with human actions, which meant they were very slow against the FPGA, and these nanosecond delays did not really matter much. Additionally, building a custom enclosure for the Basys3 board and cable mess helped me establish a professional and user-friendly design appearance for the project. In conclusion, this project overall provided a comprehensive understanding of the combination of VHDL and real-life components like an ultrasonic sensor. Moreover, it connected the theoretical Boolean algebra with the concept of finite state machines (FSM) in a practical way. The final system functioned flawlessly, and it gave a practical solution to a common health problem.

Video Link: https://youtu.be/zX3cPm8u5OE

Appendices

top_module.vhd

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;



entity top_module is

   Port ( system_clock      : in STD_LOGIC;

          ldr_input         : in STD_LOGIC;

          sensor_echo       : in STD_LOGIC;

          button_up         : in STD_LOGIC;

          button_down       : in STD_LOGIC;
```

```
        button_left       : in STD_LOGIC;
        sensor_trigger    : out STD_LOGIC;
        alarm_buzzer      : out STD_LOGIC;
        alarm_led         : out STD_LOGIC;
        ldr_status_led    : out STD_LOGIC;
        ext_countdown_led  : out STD_LOGIC;
        segment_out       : out STD_LOGIC_VECTOR (6 downto 0);
        anode_out         : out STD_LOGIC_VECTOR (3 downto 0));
end top_module;

architecture Behavioral of top_module is

   component clock_divider
      Port ( system_clock : in STD_LOGIC;
           tick_1us     : out STD_LOGIC;
           tick_1sec    : out STD_LOGIC);
   end component;

   component ultrasonic_controller
      Port ( system_clock   : in STD_LOGIC;
           tick_1us      : in STD_LOGIC;
           sensor_echo    : in STD_LOGIC;
           sensor_trigger : out STD_LOGIC;
```

```
        distance_out   : out integer range 0 to 999);
  end component;

  component master_fsm
    Port ( system_clock     : in STD_LOGIC;
        tick_1sec       : in STD_LOGIC;
        distance_in     : in integer range 0 to 999;
        ldr_input       : in STD_LOGIC;
        button_up       : in STD_LOGIC;
        button_down     : in STD_LOGIC;
        button_left     : in STD_LOGIC;
        display_value   : out integer range 0 to 999;
        buzzer_out      : out STD_LOGIC;
        alarm_led       : out STD_LOGIC;
        ext_countdown_led : out STD_LOGIC);
  end component;

  component seven_segment_driver
    Port ( system_clock : in STD_LOGIC;
        distance_in  : in integer range 0 to 999;
        segment_out  : out STD_LOGIC_VECTOR (6 downto 0);
        anode_out    : out STD_LOGIC_VECTOR (3 downto 0));
  end component;
```

```
    signal tick_1us_sig   : STD_LOGIC;
    signal tick_1sec_sig  : STD_LOGIC;
    signal raw_dist_sig   : integer range 0 to 999;
    signal final_disp_sig : integer range 0 to 999;

begin
    ldr_status_led <= ldr_input;

    Inst_clock_divider: clock_divider port map(
        system_clock => system_clock,
        tick_1us     => tick_1us_sig,
        tick_1sec    => tick_1sec_sig
    );

    Inst_ultrasonic: ultrasonic_controller port map(
        system_clock   => system_clock,
        tick_1us       => tick_1us_sig,
        sensor_echo    => sensor_echo,
        sensor_trigger => sensor_trigger,
        distance_out   => raw_dist_sig
    );
```

```
    Inst_master_fsm: master_fsm port map(
        system_clock      => system_clock,
        tick_1sec         => tick_1sec_sig,
        distance_in       => raw_dist_sig,
        ldr_input         => ldr_input,
        button_up         => button_up,
        button_down       => button_down,
        button_left       => button_left,
        display_value     => final_disp_sig,
        buzzer_out        => alarm_buzzer,
        alarm_led         => alarm_led,
        ext_countdown_led => ext_countdown_led
    );


    Inst_display: seven_segment_driver port map(
        system_clock => system_clock,
        distance_in  => final_disp_sig,
        segment_out  => segment_out,
        anode_out    => anode_out
    );


end Behavioral;
```

clock_divider.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity clock_divider is
    Port ( system_clock : in STD_LOGIC;
           tick_1us : out STD_LOGIC;
           tick_1sec : out STD_LOGIC);
end clock_divider;


architecture Behavioral of clock_divider is
    signal counter_1us : integer range 0 to 99 := 0;
    signal counter_1sec : integer range 0 to 99999999 := 0;
begin
    process(system_clock)
    begin
        if rising_edge(system_clock) then


            if counter_1us = 99 then
                counter_1us <= 0;
                tick_1us <= '1';
            else
                counter_1us <= counter_1us + 1;
```

```
            tick_1us <= '0';
        end if;



        if counter_1sec = 99999999 then
            counter_1sec <= 0;
            tick_1sec <= '1';
        else
            counter_1sec <= counter_1sec + 1;
            tick_1sec <= '0';
        end if;
      end if;
  end process;
end Behavioral;
```

ultrasonic_controller.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ultrasonic_controller is
  Port ( system_clock : in STD_LOGIC;
         tick_1us : in STD_LOGIC;
```

```
        sensor_echo : in STD_LOGIC;
        sensor_trigger : out STD_LOGIC;
        distance_out : out integer range 0 to 999);
end ultrasonic_controller;

architecture Behavioral of ultrasonic_controller is
    type state_type is (WAIT_STATE, TRIGGER_STATE, MEASURE_STATE, CALC_STATE);
    signal current_state : state_type := WAIT_STATE;

    signal timer : integer := 0;
    signal echo_time : integer := 0;
begin
    process(system_clock)
    begin
        if rising_edge(system_clock) then
            if tick_1us = '1' then
                case current_state is
                    when WAIT_STATE =>
                        sensor_trigger <= '0';
                        if timer >= 60000 then
                            timer <= 0;
                            current_state <= TRIGGER_STATE;
                        else
```

```
            timer <= timer + 1;
        end if;

    when TRIGGER_STATE =>
        sensor_trigger <= '1';
        if timer >= 10 then
            timer <= 0;
            current_state <= MEASURE_STATE;
        else
            timer <= timer + 1;
        end if;

    when MEASURE_STATE =>
        sensor_trigger <= '0';
        if sensor_echo = '1' then
            echo_time <= echo_time + 1;
        elsif echo_time > 0 and sensor_echo = '0' then
            current_state <= CALC_STATE;
        end if;

        if echo_time > 30000 then
            current_state <= CALC_STATE;
        end if;
```

```
                when CALC_STATE =>
                    distance_out <= echo_time / 58;
                    echo_time <= 0;
                    current_state <= WAIT_STATE;
                end case;
            end if;
        end if;
    end process;
end Behavioral;
```

master_fsm.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity master_fsm is
    Port ( system_clock     : in STD_LOGIC;
           tick_1sec        : in STD_LOGIC;
           distance_in      : in integer range 0 to 999;
           ldr_input        : in STD_LOGIC;
           button_up        : in STD_LOGIC;
           button_down      : in STD_LOGIC;
```

```
        button_left       : in STD_LOGIC;
        display_value     : out integer range 0 to 999;
        buzzer_out        : out STD_LOGIC;
        alarm_led         : out STD_LOGIC;
        ext_countdown_led : out STD_LOGIC);
end master_fsm;

architecture Behavioral of master_fsm is
   type state_type is (SAFE, COUNTDOWN, LOCKED);
   signal current_state : state_type := SAFE;

   signal timer : integer range 0 to 10 := 0;
   signal password_step : integer range 0 to 3 := 0;

   signal up_prev, down_prev, left_prev : STD_LOGIC := '0';
begin
   process(system_clock)
   begin
      if rising_edge(system_clock) then
         up_prev <= button_up;
         down_prev <= button_down;
         left_prev <= button_left;
```

```
        case current_state is
            when SAFE =>
                buzzer_out <= '0';
                alarm_led <= '0';
                ext_countdown_led <= '0';
                display_value <= distance_in;
                password_step <= 0;

                if distance_in < 40 and distance_in > 0 then
                    current_state <= COUNTDOWN;
                    if ldr_input = '1' then
                        timer <= 10;
                    else
                        timer <= 5;
                    end if;
                end if;

            when COUNTDOWN =>
                display_value <= timer;
                buzzer_out <= '0';
                alarm_led <= '0';
                ext_countdown_led <= '1';
```

```
            if distance_in >= 40 then
                current_state <= SAFE;
            end if;

            if tick_1sec = '1' then
                if timer > 0 then
                    timer <= timer - 1;
                else
                    current_state <= LOCKED;
                end if;
            end if;

        when LOCKED =>
            display_value <= 0;
            buzzer_out <= '1';
            alarm_led <= '1';
            ext_countdown_led <= '0';

            if (button_up = '1' and up_prev = '0') then
                password_step <= 1;
            elsif (button_down = '1' and down_prev = '0') then
                if password_step = 1 then
                    password_step <= 2;
```

```
                else
                    password_step <= 0;
                end if;
            elsif (button_left = '1' and left_prev = '0') then
                if password_step = 2 then
                    current_state <= SAFE;
                    password_step <= 0;
                else
                    password_step <= 0;
                end if;
            end if;

        end case;
    end if;
  end process;
end Behavioral;
```

seven_segment_driver.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity seven_segment_driver is
```

```
    Port ( system_clock : in STD_LOGIC;
           distance_in : in integer range 0 to 999;
           segment_out : out STD_LOGIC_VECTOR (6 downto 0);
           anode_out : out STD_LOGIC_VECTOR (3 downto 0));
end seven_segment_driver;

architecture Behavioral of seven_segment_driver is
    signal refresh_counter : unsigned(19 downto 0) := (others => '0');
    signal active_digit : STD_LOGIC_VECTOR(1 downto 0);
    signal bcd_digit : integer range 0 to 9;

    signal hundreds : integer range 0 to 9;
    signal tens : integer range 0 to 9;
    signal ones : integer range 0 to 9;
begin

    hundreds <= distance_in / 100;
    tens <= (distance_in mod 100) / 10;
    ones <= distance_in mod 10;


    process(system_clock)
    begin
```

```
    if rising_edge(system_clock) then
        refresh_counter <= refresh_counter + 1;
    end if;
end process;

active_digit <= std_logic_vector(refresh_counter(19 downto 18));



process(active_digit, hundreds, tens, ones)
begin
    case active_digit is
        when "00" =>
            anode_out <= "1110";
            bcd_digit <= ones;
        when "01" =>
            anode_out <= "1101";
            bcd_digit <= tens;
        when "10" =>
            anode_out <= "1011";
            bcd_digit <= hundreds;
        when others =>
            anode_out <= "1111";
            bcd_digit <= 0;
```

```
        end case;
    end process;



    process(bcd_digit)
    begin
        case bcd_digit is
            when 0 => segment_out <= "1000000";
            when 1 => segment_out <= "1111001";
            when 2 => segment_out <= "0100100";
            when 3 => segment_out <= "0110000";
            when 4 => segment_out <= "0011001";
            when 5 => segment_out <= "0010010";
            when 6 => segment_out <= "0000010";
            when 7 => segment_out <= "1111000";
            when 8 => segment_out <= "0000000";
            when 9 => segment_out <= "0010000";
            when others => segment_out <= "1111111";
        end case;
    end process;
end Behavioral;
```

constrains.xdc

```
set_property PACKAGE_PIN W5 [get_ports system_clock]

set_property IOSTANDARD LVCMOS33 [get_ports system_clock]

create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports system_clock]


set_property PACKAGE_PIN J1 [get_ports ldr_input]

set_property IOSTANDARD LVCMOS33 [get_ports ldr_input]


set_property PACKAGE_PIN L2 [get_ports sensor_trigger]

set_property IOSTANDARD LVCMOS33 [get_ports sensor_trigger]


set_property PACKAGE_PIN J2 [get_ports sensor_echo]

set_property IOSTANDARD LVCMOS33 [get_ports sensor_echo]


set_property PACKAGE_PIN G2 [get_ports alarm_buzzer]

set_property IOSTANDARD LVCMOS33 [get_ports alarm_buzzer]


set_property PACKAGE_PIN U16 [get_ports alarm_led]

set_property IOSTANDARD LVCMOS33 [get_ports alarm_led]


set_property PACKAGE_PIN E19 [get_ports ldr_status_led]

set_property IOSTANDARD LVCMOS33 [get_ports ldr_status_led]


set_property PACKAGE_PIN T18 [get_ports button_up]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports button_up]

set_property PACKAGE_PIN U17 [get_ports button_down]

set_property IOSTANDARD LVCMOS33 [get_ports button_down]

set_property PACKAGE_PIN W19 [get_ports button_left]

set_property IOSTANDARD LVCMOS33 [get_ports button_left]


set_property PACKAGE_PIN U7 [get_ports {segment_out[6]}]

set_property IOSTANDARD LVCMOS33 [get_ports {segment_out[6]}]

set_property PACKAGE_PIN V5 [get_ports {segment_out[5]}]

set_property IOSTANDARD LVCMOS33 [get_ports {segment_out[5]}]

set_property PACKAGE_PIN U5 [get_ports {segment_out[4]}]

set_property IOSTANDARD LVCMOS33 [get_ports {segment_out[4]}]

set_property PACKAGE_PIN V8 [get_ports {segment_out[3]}]

set_property IOSTANDARD LVCMOS33 [get_ports {segment_out[3]}]

set_property PACKAGE_PIN U8 [get_ports {segment_out[2]}]

set_property IOSTANDARD LVCMOS33 [get_ports {segment_out[2]}]

set_property PACKAGE_PIN W6 [get_ports {segment_out[1]}]

set_property IOSTANDARD LVCMOS33 [get_ports {segment_out[1]}]

set_property PACKAGE_PIN W7 [get_ports {segment_out[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {segment_out[0]}]


set_property PACKAGE_PIN U2 [get_ports {anode_out[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {anode_out[0]}]
```

```
set_property PACKAGE_PIN U4 [get_ports {anode_out[1]}]

set_property IOSTANDARD LVCMOS33 [get_ports {anode_out[1]}]

set_property PACKAGE_PIN V4 [get_ports {anode_out[2]}]

set_property IOSTANDARD LVCMOS33 [get_ports {anode_out[2]}]

set_property PACKAGE_PIN W4 [get_ports {anode_out[3]}]

set_property IOSTANDARD LVCMOS33 [get_ports {anode_out[3]}]


set_property PACKAGE_PIN H1 [get_ports ext_countdown_led]

set_property IOSTANDARD LVCMOS33 [get_ports ext_countdown_led]
```

tb_top_module.vhd

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;


entity tb_top_module is


end tb_top_module;


architecture behavior of tb_top_module is


   component top_module

   Port ( system_clock       : in STD_LOGIC;
```

```
        ldr_input         : in STD_LOGIC;
        sensor_echo       : in STD_LOGIC;
        button_up         : in STD_LOGIC;
        button_down       : in STD_LOGIC;
        button_left       : in STD_LOGIC;
        sensor_trigger    : out STD_LOGIC;
        alarm_buzzer      : out STD_LOGIC;
        alarm_led         : out STD_LOGIC;
        ldr_status_led    : out STD_LOGIC;
        ext_countdown_led  : out STD_LOGIC;
        segment_out       : out STD_LOGIC_VECTOR (6 downto 0);
        anode_out         : out STD_LOGIC_VECTOR (3 downto 0));
    end component;

    signal system_clock : std_logic := '0';
    signal ldr_input    : std_logic := '0';
    signal sensor_echo  : std_logic := '0';
    signal button_up    : std_logic := '0';
    signal button_down  : std_logic := '0';
    signal button_left  : std_logic := '0';

    -- Çıkış Sinyalleri
    signal sensor_trigger    : std_logic;
```

```
    signal alarm_buzzer      : std_logic;
    signal alarm_led         : std_logic;
    signal ldr_status_led    : std_logic;
    signal ext_countdown_led : std_logic;
    signal segment_out       : std_logic_vector(6 downto 0);
    signal anode_out         : std_logic_vector(3 downto 0);

    constant clock_period : time := 10 ns;

begin

    uut: top_module port map (
        system_clock     => system_clock,
        ldr_input        => ldr_input,
        sensor_echo      => sensor_echo,
        button_up        => button_up,
        button_down      => button_down,
        button_left      => button_left,
        sensor_trigger   => sensor_trigger,
        alarm_buzzer     => alarm_buzzer,
        alarm_led        => alarm_led,
        ldr_status_led   => ldr_status_led,
        ext_countdown_led => ext_countdown_led,
```

```
        segment_out      => segment_out,
        anode_out        => anode_out
    );

    clock_process :process
    begin
        system_clock <= '0';
        wait for clock_period/2;
        system_clock <= '1';
        wait for clock_period/2;
    end process;

    echo_process :process
    begin
        wait until rising_edge(sensor_trigger);
        wait for 10 us;

        sensor_echo <= '1';

        wait for 1740 us;

        sensor_echo <= '0';
        wait for 50 ms;
```

```
    end process;



    stim_proc: process
    begin

        ldr_input <= '1';
        button_up <= '0';
        button_down <= '0';
        button_left <= '0';

        wait for 5 ms;

        wait;
    end process;

end behavior;
```

References

[1] Bilkent University, Department of Electrical and Electronics Engineering. "EEE 102 Term Project: Smart Posture Monitor and AFK Alert System," EE102 Course Manual, Spring 2026.

[2] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with VHDL Design*, 3rd ed. New York, NY: McGraw-Hill Education, 2008.

[3] M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6th ed. Pearson, 2012.

[4] Digilent Inc., "Basys 3 FPGA Board Reference Manual." [Online]. Available: https://digilent.com/reference/programmable-logic/basys-3/reference-manual

[5] Ojanster Robotics, "HC-SR04 Ultrasonic Sensor Datasheet and Timing Diagram." [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[6] Google, "Gemini Large Language Model." Used as an AI assistant for VHDL debugging, mathematical verification, and testbench generation. [Online]. Available: https://gemini.google.com/

[7] Xilinx Inc., "Vivado Design Suite User Guide: Logic Simulation (UG900)." [Online]. Available: https://docs.xilinx.com/

[8] Digilent Inc., "Basys 3 Master XDC Constraint File," GitHub Repository. [Online]. Available: https://github.com/Digilent/digilent-xdc

[9] R. Merrick, "VHDL Tutorials and Examples: Finite State Machines," Nandland. [Online]. Available: https://nandland.com/vhdl-tutorials/

[10] IEEE Computer Society, "IEEE Standard VHDL Language Reference Manual," IEEE Std 1076-2008, 2009.